Language-Conditioned Basket Sorting with a Panda Arm in MuJoCo

Hui Xu
ESE 564 Final Project
hui.xu.3@stonybrook.edu

Abstract

This project implements a language-conditioned tabletop sorting system for a Franka Panda-style robot arm in MuJoCo. The task is to move one of two YCB objects, a red cracker box or a yellow mustard bottle, into one of two colored baskets according to a natural-language command supplied at run time. The final system combines a lightweight language parser, color-based camera perception, a finite-state pushing policy, a continuity-aware differential inverse kinematics controller, and a MuJoCo scene built from course Panda assets and public YCB meshes. The final perception-and-pushing executor solved all five required video tasks and achieved 20/20 successes on a randomized smoke evaluation. I also added a separate contact-only configuration that disables the planar proxy and achieved 18/20 successes in a randomized contact-rich evaluation. As an exploratory extension, I implemented a task-and-motion-planning grasp pipeline with RRT-Connect workspace planning and a MuJoCo contact gripper; it achieved 5/5 successes in a randomized contact-rich grasp run. The video artifacts include on-frame language captions to show the command being executed.

1 Introduction

Language-conditioned manipulation is useful because it separates the user's goal from the robot's low-level control interface. Instead of directly specifying joint targets, the user can say "place the mustard bottle in the right basket" and the robot must infer both the target object and the target location. This project focuses on a compact version of that problem: sorting two tabletop objects into two baskets using a Panda arm.

The original project proposal considered an end-effector waypoint pipeline followed by inverse kinematics and control. A concern with that approach is that independently solving inverse kinematics for consecutive end-effector targets can produce discontinuous joint-space motions, for example switching between elbow-up and elbow-down solutions for nearby Cartesian poses. To avoid that failure mode, the final implementation uses differential inverse kinematics seeded by the current joint state. Each step computes a small joint update from the local Jacobian, clips large joint jumps, and respects joint limits. This keeps consecutive actions compatible with each other while preserving a simple Cartesian policy interface.

The final system is intentionally pragmatic. The policy used for final evaluation is a model-based finite-state machine (FSM), not a learned vision-language policy. I also implemented demonstration collection and a simple behavior cloning baseline to keep a learning path available, but the learned baseline was not strong enough to use as the final executor. The final submitted path uses camera-derived object estimates and MuJoCo-integrated planar pushing, which better matches the assignment requirements than the earlier privileged-pose pick-and-place prototype. After building the pushing system, I added a separate grasping extension based on task and motion planning ideas; it is reported as an experimental result rather than replacing the more validated contact-push evidence.

Preprint. Under review.

2 Task and assets

The task scene contains a Panda arm, two movable objects, and two baskets. The class Panda scene is located at `assets/class_panda/basket_scene.mjcf`. It uses the homework Panda MJCF and mesh assets extracted from the course homework zip, plus public YCB Google 16k meshes for the cracker box and mustard bottle. The color convention used in the final demo is shown in Table 1.

Table 1: Objects and baskets in the final MuJoCo scene.

Entity	Visual color	Semantic label
YCB cracker box	Red	`cracker_box`
YCB mustard bottle	Yellow	`mustard_bottle`
Left basket	Blue	`left` basket
Right basket	Green	`right` basket

Although the cleanest demo convention is red/cracker to blue/left and yellow/mustard to green/right, evaluation is not hard-coded to that mapping. The environment samples language commands that may request either object and either basket. This tests whether the decision pipeline follows the command rather than memorizing a fixed visual color rule.

3 Method

3.1 System overview

The robot decision-making pipeline has six main components:

1. A language parser extracts the target object, target basket, and speed modifier from the instruction.
2. A color-based perception module estimates the red and yellow object positions from rendered RGB frames.
3. A scripted pushing FSM converts the parsed task and perceived object position into approach, lower, push, and retreat waypoints.
4. The experimental grasp extension uses TAMP-style grasp sampling plus RRT-Connect workspace planning for approach and transfer waypoints.
5. A differential IK controller converts each Cartesian target into a continuous joint-space update.
6. The MuJoCo environment integrates object motion during the push phase and checks success after the commanded object enters the goal basket region.

All policies use the same action interface:

$$a_t = [x_t, y_t, z_t, g_t],$$

where (x_t, y_t, z_t) is the desired end-effector target and g_t is the gripper command. Positive gripper values mean open, and negative values mean close. This interface lets the same FSM and behavior-cloning code run in both the lightweight kinematic fallback environment and the MuJoCo class scene.

3.2 Language parsing

The parser is rule-based because the command grammar is intentionally small. It detects object words such as “cracker,” “box,” “mustard,” and “bottle,” basket words such as “left” and “right,” and optional speed words such as “careful” or “fast.” The parsed task is stored as a structured task specification. This is enough for the current task distribution, and it makes evaluation deterministic and easy to debug.

At run time, the user can provide the language instruction directly through the command-line interface. For example, the following command makes the robot repeatedly execute the requested red/cracker to blue/left task under randomized object starts:

```
python scripts/evaluate.py --config configs/class_panda.yaml --policy push_fsm --
    instruction "place the cracker box in the left basket" --episodes 3
```

Changing only the instruction string changes the selected object and basket:

```
python scripts/evaluate.py --config configs/class_panda.yaml --policy push_fsm --
    instruction "place the mustard bottle in the right basket" --episodes 3
```

To make this visible in the submitted artifacts, the video writer overlays the executed command on every saved GIF frame with a caption beginning with "Language:". The evaluation JSON also stores the same string in each episode's `instruction` field. This means the video itself shows which language command was entered, while the log file provides a machine-readable record of the same command.

3.3 Perception-driven pushing policy

The final FSM policy is implemented in `src/basket_sorting/push_fsm.py`. Given the parsed language command and the perceived position of the selected object, it computes a push line from behind the object toward the requested basket. The phases are:

$$\texttt{approach\_push\_start} \rightarrow \texttt{lower\_to\_push} \rightarrow \texttt{push} \rightarrow \texttt{retreat}.$$

The FSM switches phases when the end-effector is close enough to the current waypoint or after the push has been applied for a fixed number of control steps. Speed words modify the commanded motion step size; for example, "careful" moves more slowly than "fast." The final policy uses perceived object positions for the variable part of the scene and fixed basket centers as known goal regions.

3.4 Color perception

The perception module is implemented in `src/basket_sorting/perception.py`. It segments rendered RGB images using color thresholds for the red cracker box and yellow mustard bottle, computes the pixel centroid of each mask, and converts the centroid to an approximate tabletop coordinate with a calibrated affine image-to-world map. The controller uses these perceived object positions instead of directly reading simulator object poses. Simulator poses are still used for randomized reset and final success scoring.

3.5 Differential inverse kinematics

The controller avoids solving each IK target independently. At time t, it computes the end-effector position $p(q_t)$ and Jacobian $J(q_t)$ at the current joint configuration. For a desired Cartesian displacement Δx, it solves a damped least-squares update:

$$\Delta q = J^\top (JJ^\top + \lambda^2 I)^{-1} \Delta x.$$

The update is then clipped by a maximum joint-step threshold and by joint limits. This makes the local IK solution continuous from the previous state and reduces the risk of sudden joint discontinuities. The same controller interface is used with a toy kinematic backend for the fallback environment and a MuJoCo Jacobian backend for the class Panda scene.

3.6 Physics-based object motion

The MuJoCo environment contains free joints for the two YCB objects. The earlier prototype used an attachment rule for pick-and-place, but the final evaluation path disables that behavior and uses a pushing proxy during the `push` phase. The proxy applies planar generalized forces and velocities to the target object's free joint while MuJoCo integrates the object's motion. The baskets are visual goal regions, so their walls do not block the pushed object from entering the target region.

The repository also includes an experimental contact-only configuration in `configs/class_panda_contact.yaml`. That path disables the planar proxy, uses simple object collision primitives with

visible color markers, localizes objects from an overhead perception camera, disables unrelated Panda link collisions, and moves the target through MuJoCo contact with a dedicated pusher body. The contact FSM uses slow quasi-static pushes, short settle phases, a lift-and-replan step between lateral and forward pushes, and contact-aware pusher offsets so the pusher center does not drive through the desired object center.

3.7 Experimental TAMP grasp extension

I also implemented a grasping extension inspired by task and motion planning (TAMP) (8), PDDLStream-style sampling of continuous parameters (9), and RRT-Connect sampling-based motion planning (10). The implementation is intentionally smaller than a full external TAMP stack such as MoveIt Task Constructor (11), but it follows the same structure: parse the language command into a symbolic goal, sample continuous grasp candidates, check simple feasibility constraints, plan collision-aware approach and transfer paths, and execute a pick-and-place skeleton.

The grasp policy is implemented in `src/basket_sorting/tamp_grasp.py` and run with `configs/class_panda_grasp.yaml`. The symbolic goal has the form `place(object, basket)`. The planner samples object-specific grasp candidates around the perceived object pose, rejects candidates outside the workspace or beyond the gripper-width limit, and then calls a project-sized RRT-Connect implementation in `src/basket_sorting/rrt.py`. This RRT planner searches in 3D end-effector workspace around inflated object AABBs, and the resulting waypoints are executed by the differential IK controller. This is not a full joint-space planner with exact mesh collision checks, but it removes the previous pure straight-line approach and transfer assumption.

$$\texttt{rrt\_approach} \rightarrow \texttt{grasp} \rightarrow \texttt{close} \rightarrow \texttt{lift} \rightarrow \texttt{rrt\_transfer} \rightarrow \texttt{place} \rightarrow \texttt{release} \rightarrow \texttt{rrt\_retreat}.$$

Future versions could replace the hand-written grasp sampler with a learned grasp scorer such as DexNet/GQ-CNN (13) or Contact-GraspNet (14), while keeping the same symbolic goal and execution skeleton.

This extension is useful because it makes the system architecture closer to a standard pick-and-place manipulation planner. However, I keep it separate from the main pushing result because it was added late in the project. The grasp configuration disables the manual attachment fallback by enabling a dedicated MuJoCo grasp tool. The tool is a regular dynamic body welded to a mocap target, with two side contact pads, a top adhesive contact pad, and a MuJoCo adhesion actuator that turns on during the `close` and transport phases. Object motion in this path comes from MuJoCo contact and adhesion forces rather than direct object pose setting. The grasp configuration also enables Panda link collision geometry against the YCB objects so that non-gripper arm links cannot pass through objects silently; this follows the planning-scene and allowed-collision-matrix idea used in MoveIt (12). The intentional allowed contacts are the grasp pads with the selected object.

3.8 Learning baseline

The repository also includes scripts for collecting FSM demonstrations and fitting a lightweight behavior cloning baseline. Demonstrations store state features, subtask text, speed, and the low-level action. The current NumPy linear behavior cloning model is included as a smoke-test baseline only. In previous testing on the class scene it achieved 0/20 successes, while the FSM achieved 20/20. Because the assignment asks for a final product that reliably solves the task, the FSM is used as the final executor.

4 Results

4.1 Evaluation protocol

The main evaluation used `configs/class_panda.yaml`. Each episode samples a random target object, target basket, speed word, and initial object placement. Success is counted when the commanded target object reaches the commanded basket region. The final larger evaluation was run with:

```
python scripts/evaluate.py --config configs/class_panda.yaml --policy push_fsm --
    episodes 20
```

The contact-rich five-task video was generated with:

```
python scripts/evaluate.py --config configs/class_panda_contact.yaml --policy
    push_fsm --episodes 5 --save-video videos/class_panda_contact_eval_5.gif --
    video-episodes 5
```

4.2 Quantitative results

Table 2 reports the final quantitative results. The 20-episode run includes both objects, both baskets, randomized object starts, and speed modifiers.

Table 2: Class Panda MuJoCo evaluation results.

Evaluation	Episodes	Successes	Success rate	Avg. steps	Step range
Five-task video run	5	5	100%	41.60	27–52
Explicit cracker-to-left command	3	3	100%	48.00	42–52
Explicit mustard-to-right command	3	3	100%	31.67	27–36
Randomized smoke evaluation	20	20	100%	38.75	25–52
Experimental contact-only gate	20	18	90%	385.25	130–900
Experimental contact-rich TAMP+RRT grasp	5	5	100%	241.60	181–271

4.3 Perception sanity check

I also measured the color-perception module independently from the control policy by randomizing the scene, rendering the configured perception camera, estimating each object's tabletop position, and comparing the estimate to the MuJoCo object pose. The stable demo-view configuration detected 199/200 object instances over 100 randomized resets, with mean tabletop error 7.04 cm and maximum error 15.49 cm. This is coarse but sufficient for the stable proxy-based push executor because the push policy uses broad approach regions.

The contact-only configuration uses an overhead camera and colored object-center markers. For detected objects, it was more accurate: mean error 2.11 cm and maximum error 6.03 cm. However, it detected only 131/200 object instances because the arm can occlude the overhead markers at some reset poses. This explains why the contact-only policy is more physically faithful but still less reliable than the proxy baseline.

4.4 Qualitative results

Figure 1 shows snapshots from the generated contact-rich five-task pushing video. The sequence illustrates the robot observing two objects, moving through the contact pushing behavior, and the target object entering the requested basket region. The full video file is included at `videos/class_panda_contact_eval_5.mp4`, with the GIF version at `videos/class_panda_contact_eval_5.gif`.

I also include two instruction-specific videos to make the language interface explicit: `videos/push_language_red_to_blue.gif` was generated from "place the cracker box in the left basket," and `videos/push_language_yellow_to_green.gif` was generated from "place the mustard bottle in the right basket." In both cases, the GIF frames contain the command as an on-screen language caption, the printed evaluation logs show the same instruction string for each episode, and the robot acts on that specified object-basket pair. Since most PDF viewers do not reliably play animated GIFs, the report includes still frames from these GIFs in Figure 2, while the submission zip includes the full animated GIF files.

The experimental TAMP grasp extension has its own video artifact at `videos/class_panda_grasp_collision_eval_5.mp4`, with the GIF version at `videos/class_panda_grasp_collision_eval_5.gif`. Figure 3 shows still frames from a cracker-box grasp episode after applying the visual hand offset. The contact interaction is generated by the MuJoCo gripper pad geoms, while the rendered Panda hand mesh is kept above and behind the box so the figure does not imply that the arm is passing through the target.

Figure 1: Snapshots from the five-task contact-rich MuJoCo pushing video artifact. The camera view shows the language command caption, the two colored YCB objects, the basket regions, and the contact pusher moving objects through MuJoCo contact.

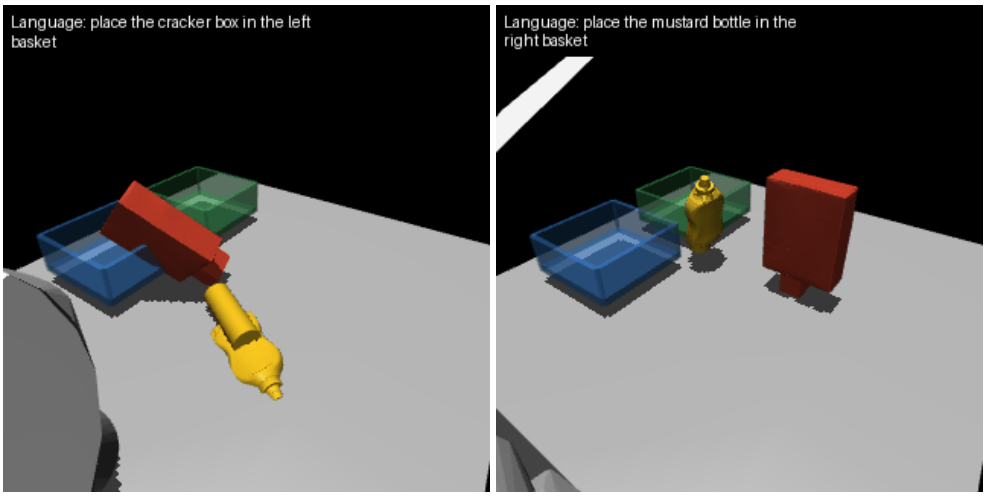

Figure 2: Still frames from the two explicit language-command GIFs. The caption at the top of each frame is the command provided through the command-line interface, making the language input visible in the report PDF as well as in the video artifacts.

4.5 Discussion and limitations

The main result is that the model-based perception-and-pushing executor solves the required five video tasks and succeeds on most randomized trials. The use of differential IK made the Cartesian waypoint policy smooth enough for repeated trials, and the language parser was sufficient for the command set used in this project.

The main limitation is that perception is a simple calibrated color-segmentation system, so it depends on object colors and camera placement. The stable default pushing dynamics still use a planar MuJoCo force/velocity proxy during the push phase. The new contact-only path is more physically faithful and passes the larger 18/20 randomized gate, but it is much slower and still less reliable than the proxy baseline. The TAMP grasp extension demonstrates a cleaner grasp-and-place decision

Figure 3: Still frames from the experimental contact-rich TAMP grasp video with Panda link/object collision enabled. This cracker-box sequence uses the corrected visualization: the rendered Panda hand is offset above/behind the box, and the MuJoCo contact gripper pads are the bodies that close on, lift, and transport the target.

structure and now uses MuJoCo contact/adhesion rather than a manual attachment rule. It also enables Panda link/object collision checks and RRT-Connect workspace planning in the grasp configuration, but it remains a simplified primitive-style collision model and adhesive gripper, not a fully tuned Franka finger contact model with a full joint-space sampling-based planner. The behavior cloning baseline remains weak because the simple linear policy does not capture the phase-dependent structure of the task. A stronger learned version would use an image encoder, language embeddings, and a recurrent or transformer policy, while retaining the current differential IK controller as a low-level action interface.

5 Code and reproducibility

The repository includes a README with installation and run instructions. The shortest setup path is:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[mujoco]"
.\.venv\Scripts\python.exe scripts/evaluate.py --config configs/
    class_panda_contact.yaml --policy push_fsm --episodes 5 --save-video videos/
    class_panda_contact_eval_5.gif --video-episodes 5
.\.venv\Scripts\python.exe scripts\evaluate.py --config configs/class_panda.yaml
     --policy push_fsm --instruction "place the mustard bottle in the right
    basket" --episodes 3 --save-video videos/push_language_yellow_to_green.gif
.\.venv\Scripts\python.exe scripts\evaluate.py --config configs/
    class_panda_grasp.yaml --policy tamp_grasp --episodes 5 --save-video videos/
    class_panda_grasp_collision_eval_5.gif --video-episodes 5
```

The most important files are:

- `README.md`: installation and evaluation instructions.
- `configs/class_panda.yaml`: final class Panda scene configuration.
- `configs/class_panda_contact.yaml`: experimental contact-only class Panda configuration.
- `configs/class_panda_grasp.yaml`: experimental TAMP+RRT grasp configuration.
- `assets/class_panda/basket_scene.mjcf`: MuJoCo scene with Panda, YCB objects, baskets, and demo camera.
- `src/basket_sorting/perception.py`: color-based object perception.
- `src/basket_sorting/push_fsm.py`: final scripted pushing policy.
- `src/basket_sorting/tamp_grasp.py`: experimental TAMP grasp planner and policy that calls RRT-Connect for approach, transfer, and retreat paths.
- `src/basket_sorting/rrt.py`: RRT-Connect end-effector workspace planner used by the grasp extension.
- `src/basket_sorting/envs/mujoco_env.py`: MuJoCo environment and differential IK integration.
- `scripts/evaluate.py`: evaluation entry point.
- `scripts/perception_sanity.py`: perception localization metric.

6 References

References

[1] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

[2] Berk Calli, Arjun Singh, Aaron Walsman, Siddhartha Srinivasa, Pieter Abbeel, and Aaron M. Dollar. The YCB object and model set: Towards common benchmarks for manipulation research. In *International Conference on Advanced Robotics*, 2015.

[3] Bruno Siciliano, Lorenzo Sciavicco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2010.

[4] Franka Emika. Panda robot documentation and model resources. `https://franka.de/`.

[5] Google DeepMind. MuJoCo Menagerie robot model collection. `https://github.com/google-deepmind/mujoco_menagerie`.

[6] Kevin M. Lynch and Matthew T. Mason. Stable pushing: Mechanics, controllability, and planning. *The International Journal of Robotics Research*, 1996.

[7] Google DeepMind. MuJoCo modeling and contact documentation. `https://mujoco.readthedocs.io/`.

[8] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomas Lozano-Perez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 2021.

[9] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *International Conference on Automated Planning and Scheduling*, 2020.

[10] James J. Kuffner and Steven M. LaValle. RRT-Connect: An efficient approach to single-query path planning. In *IEEE International Conference on Robotics and Automation*, 2000.

[11] MoveIt. MoveIt Task Constructor pick-and-place tutorial. `https://moveit.github.io/moveit_task_constructor/tutorials/pick-and-place.html`.

[12] MoveIt. MoveIt concepts: Planning scene and collision checking. `https://moveit.ai/documentation/concepts/`.

[13] Jeffrey Mahler et al. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. `https://bair.berkeley.edu/blog/2017/06/27/dexnet-2.0/`.

[14] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. Contact-GraspNet: Efficient 6-DoF grasp generation in cluttered scenes. `https://research.nvidia.com/publication/2021-03_contact-graspnet-efficient-6-dof-grasp-generation-cluttered-scenes`.